Lecture 5 — Flows II: Min-Cost Max-Flow, Hopcroft–Karp, Min-Cut Modeling, Lower Bounds

> National Championship Prep · Prerequisites: Network flow I (week 32: Edmonds–Karp, Dinic, min cut, bipartite matching), shortest paths (week 17), graph modeling (regional-04) · Estimated time: 2 × 90 min

1. Overview

Week 32 established max-flow = min-cut and used Dinic for bipartite matching. At championship level, flow problems are rarely "here is a network, compute the flow". The difficulty is **modeling**, and the model often needs one of four extensions:

Extension	When the statement says…
Min-cost max-flow	"minimize total cost", "assign … at minimum price", "k disjoint paths of minimum total length"
Hopcroft–Karp	large bipartite matching (10^5 vertices), vertex covers, independent sets, path covers
Min-cut modeling	"choose a subset" with dependencies, "each item goes to side A or B with penalties"
Lower bounds	"each edge must carry at least l", "every road must be patrolled", "between l and r items of each type"

Flow problems at this level usually have small graphs (10^3–10^4 vertices) and the time limit is set for Dinic or SSP min-cost flow; practical running time is far below the worst-case bounds. Your notebook (Lecture 1) should contain Dinic, MCMF and Hopcroft–Karp, each judge-tested.

2. Min-cost max-flow

2.1 Problem

Each edge has capacity c(e) and cost w(e) per unit of flow. Among all maximum flows (or all flows of value exactly k), find one minimizing $\Sigma\, w(e)\cdot f(e)$.

2.2 Successive shortest paths (SSP)

Repeat: find a **cheapest** s–t path in the residual graph (reverse edges have cost −w), push as much flow as possible along it.

Optimality criterion. A flow f of value v is min-cost among flows of value v if and only if the residual graph has **no negative-cost cycle**. (If it had one, pushing flow around it keeps the value and lowers the cost; conversely, the difference of two flows of equal value decomposes into cycles in the residual graph of the first, so a cheaper flow implies a negative cycle.)

SSP preserves the criterion. If the residual graph of f has no negative cycle and we augment along a shortest path, the new residual graph has none either (shortest-path distances give a feasible potential before and after the augmentation, see below). By induction from the zero flow (valid if the original graph has no negative cycles), every intermediate flow is min-cost for its value. Stopping after value k gives the min-cost flow of value k; stopping when no path exists gives min-cost max-flow.

Min-cost flow of any value. If you want the cheapest flow regardless of value (e.g. costs can be negative "profits"), stop as soon as the shortest path has cost ≥ 0: path costs are non-decreasing across iterations.

2.3 Potentials: Dijkstra on negative edges

Reverse edges have negative cost, so plain Dijkstra does not apply. Keep a **potential** π(v) and use **reduced costs** $w_\pi(u, v) = w(u, v) + \pi(u) - \pi(v)$. Along any s–t path, reduced cost = real cost + π(s) − π(t), so shortest paths are the same.

If π(v) = dist(s, v) (shortest distance in the current residual graph), every residual edge has $w_\pi \geq 0$ (triangle inequality), and edges on shortest paths have $w_\pi = 0$. After augmenting along a shortest path, the only new residual edges are reverses of path edges, whose reduced cost is −0 = 0. So **after each Dijkstra, set π(v) += dist_π(v)** and all reduced costs remain non-negative for the next round (Johnson's technique).

Initial potentials: all zeros if all costs are ≥ 0; otherwise one Bellman–Ford/SPFA from s (requires no negative cycles; a DAG is the common case).

Complexity. O(F · m log n) with F = total flow, plus one Bellman–Ford. F can be large; in practice SSP is fast because each augmentation saturates an edge and the number of distinct path lengths is small. When F is huge and capacities are large, capacity scaling or network simplex exists, but contest problems almost never require it.

2.4 Modeling patterns

- **Assignment problem** (n workers × n jobs, cost matrix): source → worker (cap 1, cost 0), worker → job (cap 1, cost c), job → sink (cap 1, cost 0). CSES 2129 — Task Assignment. (The Hungarian algorithm is $O(n^3)$ and is covered in worldfinals-03.)
- **k edge-disjoint paths of minimum total length**: unit capacities, cost = length, flow value k. CSES 2130 — Distinct Routes II.
- **Convex costs**: if putting the t-th unit through an "edge" costs an increasing amount (1, 3, 5, … for a cost of t^2), use parallel unit-capacity edges with costs equal to the marginal costs. Convexity guarantees the cheap copies are used first. Codeforces 863F — Almost Permutation.
- **Degree-constrained structures**: "each node has at most 2 children, minimize total edge length" → each node is split into a "child side" and a "parent side". Codeforces 277E — Binary Tree on Plane.
- **Maximum profit**: negate costs (watch for negative cycles; with a DAG they cannot occur).

3. Hopcroft–Karp

3.1 Algorithm

Kuhn's algorithm (week 32) finds one augmenting path per DFS: $O(V \cdot E)$. Hopcroft–Karp works in **phases**:

1. **BFS** from all free left vertices simultaneously, layering the graph by alternating-path distance; stop the BFS at the layer where a free right vertex is reached.
2. **DFS** from each free left vertex, only following edges to the next layer, to find a **maximal set of vertex-disjoint shortest augmenting paths**. Dead-end vertices are removed for the rest of the phase (`dist[u] = -1`), so each phase is $O(E)$.

3.2 Why O(E√V)

- Shortest augmenting path length strictly increases after each phase (standard lemma, same as Dinic's level graph argument).
- After $\sqrt{V}$ phases, every augmenting path has length $> \sqrt{V}$. Compare the current matching M with a maximum matching M*: their symmetric difference contains $|M^*| - |M|$ vertex-disjoint augmenting paths, each with $> \sqrt{V}$ vertices, so $|M^*| - |M| < \sqrt{V}$. Each further phase adds ≥ 1 edge, so at most $\sqrt{V}$ more phases.

Total $O(\sqrt{V})$ phases × $O(E)$ = **$O(E\sqrt{V})$**. Dinic on the unit-capacity bipartite network has the same bound; Hopcroft–Karp has a smaller constant and is shorter.

3.3 What matchings give you (bipartite graphs)

- **König's theorem:** minimum vertex cover = maximum matching. Construct the cover from the final BFS: let Z = vertices reachable from free left vertices by alternating paths; cover = (L \ Z) ∪ (R ∩ Z).
- **Maximum independent set** = V − maximum matching (complement of a minimum vertex cover). Classic in grid problems: cells colored like a chessboard, conflicts only between colors. Codeforces 1404E — Bricks is an example of the "minimum pieces = cells − max independent set of merges" form.
- **Minimum path cover in a DAG** (vertex-disjoint paths covering all vertices) = n − maximum matching in the split graph (u_out → v_in for each edge). If paths may share vertices, first take the transitive closure.
- **Hall's theorem** (a left-saturating matching exists iff every left subset S has |N(S)| ≥ |S|) is often the key to proving a greedy or a formula without running any flow.

4. Min-cut modeling

4.1 Reading a cut as a choice

An s–t cut splits vertices into S (source side) and T (sink side). Model each binary decision as "which side is vertex v on", and each penalty as an edge that is **cut exactly when the penalty applies**:

Constraint / penalty	Edge
pay a if v ∈ T ("not chosen")	s → v, capacity a
pay b if v ∈ S ("chosen")	v → t, capacity b
pay c if u ∈ S and v ∈ T	u → v, capacity c
forbid u ∈ S and v ∈ T (u chosen ⇒ v chosen)	u → v, capacity ∞

The minimum cut is the minimum total penalty. Penalties must be non-negative and of the "u ∈ S, v ∈ T" form; pairwise terms that reward *different* sides need a bipartite flip (swap the meaning of S and T on one side) and only work if the graph of such terms is bipartite.

4.2 Project selection (maximum-weight closure)

Projects give profit p_i; tools cost c_j; each project requires some tools. Maximize profit − cost.

Network: s → project (cap p_i), tool → t (cap c_j), project → required tool (cap ∞). Answer = Σ p_i − min cut. The source side of the min cut is the optimal set (a project on the source side forces its tools there, because ∞ edges cannot be cut).

Codeforces 1082G — Petya and Graph: edges are projects (profit w), vertices are tools (cost a_v). See §6.4.

4.3 Practical points

- **Vertex capacities:** split v into v_in → v_out with capacity c(v).
- **Recovering the cut:** after max-flow, the source side is the set of vertices reachable from s in the residual graph (CSES 1695 — Police Chase).
- **∞:** use a value larger than any finite cut (e.g. `LLONG_MAX / 4`), never `LLONG_MAX` itself (it overflows on addition).

5. Flows with lower bounds

Each edge e = (u, v) must carry f(e) with l(e) ≤ f(e) ≤ c(e).

5.1 Feasible circulation (no source/sink)

Force l(e) units through every edge and fix the imbalance:

1. Replace e by an edge of capacity c(e) − l(e) (the optional part).
2. Let excess(v) = Σ l(in-edges of v) − Σ l(out-edges of v).
3. Add a super source S and super sink T: S → v with capacity excess(v) if positive, v → T with capacity −excess(v) if negative.
4. A feasible circulation exists **iff** the max flow S → T saturates all S-edges (equals Σ positive excess). The actual flow on e is l(e) + (flow on the reduced edge).

Why: in the reduced network, vertex v has received excess(v) units "for free" from the forced lower bounds; S → v edges represent exactly that, and conservation in the reduced network is equivalent to conservation in the original.

5.2 Feasible and maximum s–t flow

Add an edge t → s with capacity ∞; now any s–t flow is a circulation. Run §5.1.

- The flow on t → s is the value of a feasible flow.
- **Maximum flow:** remove the t → s edge (and S, T) and run max-flow from s to t in the remaining residual graph; answer = feasible value + this extra flow.
- **Minimum flow:** instead run max-flow from **t to s** in the residual graph; answer = feasible value − that flow.

5.3 Min-cost flow with lower bounds

Same reduction with costs: forced units contribute Σ l(e)·w(e) up front; run MCMF from S to T (with the t → s edge of cost 0) and check saturation. Codeforces 1288F — Red-Blue Graph is a good exercise.

6. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall` and were stress-tested on random small graphs: MCMF against an independent Bellman–Ford SSP (including negative-cost DAG edges), Hopcroft–Karp against Kuhn's algorithm, lower bounds against exhaustive enumeration of all integer flows, and project selection against enumeration of all vertex subsets.

6.1 Min-cost max-flow (Dijkstra with potentials)

Input: `n m s t`, then m lines `u v cap cost` (0-indexed). Output: max flow and its min cost.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// Min-cost max-flow: successive shortest paths with Johnson potentials.
// Negative edge costs are allowed as long as there is no negative cycle.
// Complexity: O(F * m log n) with F = total flow (plus one Bellman-Ford).
struct MCMF {
    struct E { int to; ll cap, cost; };
    int n;
    vector<E> e;
    vector<vector<int>> g;
    vector<ll> pot, dist;
    vector<int> pe;  // edge used to reach each vertex
    MCMF(int n) : n(n), g(n), pot(n), dist(n), pe(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].push_back(e.size()); e.push_back({v, cap, cost});
        g[v].push_back(e.size()); e.push_back({u, 0, -cost});
    }
    void initPotentials(int s) {           // Bellman-Ford (SPFA) for negative costs
        const ll INF = LLONG_MAX / 4;
        fill(pot.begin(), pot.end(), INF);
        pot[s] = 0;
        vector<bool> inq(n, false);
        deque<int> q = {s};
        while (!q.empty()) {
            int u = q.front(); q.pop_front(); inq[u] = false;
            for (int id : g[u]) if (e[id].cap > 0 && pot[u] + e[id].cost < pot[e[id].to])
{
                pot[e[id].to] = pot[u] + e[id].cost;
                if (!inq[e[id].to]) { inq[e[id].to] = true; q.push_back(e[id].to); }
            }
```

```
        }
        for (auto& p : pot) if (p == INF) p = 0;  // unreachable: any value is fine
    }
    bool dijkstra(int s, int t) {
        const ll INF = LLONG_MAX / 4;
        fill(dist.begin(), dist.end(), INF);
        dist[s] = 0;
        priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq;
        pq.push({0, s});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;
            for (int id : g[u]) {
                const E& ed = e[id];
                ll nd = d + ed.cost + pot[u] - pot[ed.to];   // reduced cost >= 0
                if (ed.cap > 0 && nd < dist[ed.to]) { dist[ed.to] = nd; pe[ed.to] = id;
pq.push({nd, ed.to}); }
            }
        }
        if (dist[t] == INF) return false;
        for (int v = 0; v < n; v++) if (dist[v] < INF) pot[v] += dist[v];
        return true;
    }
    pair<ll,ll> run(int s, int t, ll maxFlow = LLONG_MAX) {
        initPotentials(s);
        ll flow = 0, cost = 0;
        while (flow < maxFlow && dijkstra(s, t)) {
            ll f = maxFlow - flow;
            for (int v = t; v != s; v = e[pe[v] ^ 1].to) f = min(f, e[pe[v]].cap);
            for (int v = t; v != s; v = e[pe[v] ^ 1].to) { e[pe[v]].cap -= f; e[pe[v] ^
1].cap += f; }
            flow += f;
            cost += f * (pot[t] - pot[s]);
        }
        return {flow, cost};
    }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, s, t; cin >> n >> m >> s >> t;     // 0-indexed vertices
    MCMF mf(n);
    for (int i = 0; i < m; i++) { int u, v; ll c, w; cin >> u >> v >> c >> w;
mf.addEdge(u, v, c, w); }
    auto [flow, cost] = mf.run(s, t);
    cout << flow << " " << cost << "\n";
}
```

`run(s, t, k)` stops after k units — the min-cost flow of value k. For "cheapest flow of any value" stop when `pot[t] − pot[s] ≥ 0`.

6.2 Hopcroft–Karp

Input: `nL nR m`, then m edges `u v` (0-indexed left, right).

```cpp
#include <bits/stdc++.h>
using namespace std;

// Hopcroft-Karp maximum bipartite matching: O(E sqrt(V)).
// Left vertices 0..nL-1, right vertices 0..nR-1.
struct HopcroftKarp {
    int nL, nR;
    vector<vector<int>> adj;
    vector<int> matchL, matchR, dist;
    HopcroftKarp(int nL, int nR) : nL(nL), nR(nR), adj(nL), matchL(nL, -1), matchR(nR,
-1), dist(nL) {}
    void addEdge(int u, int v) { adj[u].push_back(v); }
    bool bfs() {                        // layer the free left vertices at distance 0
        queue<int> q;
        bool found = false;
        for (int u = 0; u < nL; u++) {
            if (matchL[u] == -1) { dist[u] = 0; q.push(u); } else dist[u] = -1;
        }
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                int w = matchR[v];
                if (w == -1) found = true;                      // a shortest augmenting
path exists
                else if (dist[w] == -1) { dist[w] = dist[u] + 1; q.push(w); }
            }
        }
        return found;
    }
    bool dfs(int u) {                   // only follow edges that go one layer deeper
        for (int v : adj[u]) {
            int w = matchR[v];
            if (w == -1 || (dist[w] == dist[u] + 1 && dfs(w))) {
                matchL[u] = v; matchR[v] = u;
                return true;
            }
        }
        dist[u] = -1;                   // dead end: never visit again this phase
        return false;
    }
    int maxMatching() {
        int res = 0;
        while (bfs())
            for (int u = 0; u < nL; u++)
                if (matchL[u] == -1 && dfs(u)) res++;
        return res;
    }
};

int main() {
```

```
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int nL, nR, m; cin >> nL >> nR >> m;
    HopcroftKarp hk(nL, nR);
    for (int i = 0; i < m; i++) { int u, v; cin >> u >> v; hk.addEdge(u, v); }
    cout << hk.maxMatching() << "\n";
}
```

The DFS recursion depth is at most the augmenting-path length, which is $O(\sqrt{V})$ in all but the first few phases; for $V \approx 10^5$ on a judge with a small stack, an iterative DFS is safer.

6.3 Maximum s–t flow with lower bounds

Input: `n m s t`, then m lines `u v lo hi`. Output: maximum flow value or -1 if no feasible flow exists.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Dinic {
    struct E { int to; ll cap; };
    int n; vector<E> e; vector<vector<int>> g; vector<int> level, it;
    Dinic(int n) : n(n), g(n), level(n), it(n) {}
    int addEdge(int u, int v, ll cap) {
        g[u].push_back(e.size()); e.push_back({v, cap});
        g[v].push_back(e.size()); e.push_back({u, 0});
        return (int)e.size() - 2;
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1); level[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) { int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && level[e[id].to] < 0) {
level[e[id].to] = level[u] + 1; q.push(e[id].to); } }
        return level[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i]; int v = e[id].to;
            if (e[id].cap > 0 && level[v] == level[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) { fill(it.begin(), it.end(), 0); while (ll f = dfs(s, t,
LLONG_MAX)) flow += f; }
        return flow;
```

```
    }
};

// Maximum s-t flow where every edge carries between lo and hi units.
// Returns -1 if no feasible flow exists.
ll maxFlowLowerBounds(int n, int s, int t, const vector<array<ll,4>>& edges) {
    const ll INF = LLONG_MAX / 4;
    int S = n, T = n + 1;                       // super source / super sink
    Dinic d(n + 2);
    vector<ll> excess(n, 0);                    // forced inflow minus forced outflow
    for (auto [u, v, lo, hi] : edges) {
        d.addEdge(u, v, hi - lo);               // only the optional part is flexible
        excess[v] += lo; excess[u] -= lo;
    }
    ll need = 0;
    for (int v = 0; v < n; v++) {
        if (excess[v] > 0) { d.addEdge(S, v, excess[v]); need += excess[v]; }
        if (excess[v] < 0) d.addEdge(v, T, -excess[v]);
    }
    int ts = d.addEdge(t, s, INF);              // turns the s-t flow into a circulation
    if (d.maxflow(S, T) != need) return -1;     // some lower bound cannot be met
    ll flow = d.e[ts ^ 1].cap;                  // flow on t->s = value of the feasible
flow
    d.e[ts].cap = d.e[ts ^ 1].cap = 0;          // delete the t->s edge
    return flow + d.maxflow(s, t);              // push more along residual s->t paths
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m, s, t; cin >> n >> m >> s >> t;
    vector<array<ll,4>> edges(m);
    for (auto& [u, v, lo, hi] : edges) cin >> u >> v >> lo >> hi;
    cout << maxFlowLowerBounds(n, s, t, edges) << "\n";
}
```

To recover the flow on each original edge, remember the id returned by `addEdge` for that edge: flow = lo + (reverse edge's capacity).

6.4 Project selection: Petya and Graph (CF 1082G)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Dinic {
    struct E { int to; ll cap; };
    int n; vector<E> e; vector<vector<int>> g; vector<int> level, it;
    Dinic(int n) : n(n), g(n), level(n), it(n) {}
    void addEdge(int u, int v, ll cap) {
        g[u].push_back(e.size()); e.push_back({v, cap});
        g[v].push_back(e.size()); e.push_back({u, 0});
    }
```

```
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1); level[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) { int u = q.front(); q.pop();
            for (int id : g[u]) if (e[id].cap > 0 && level[e[id].to] < 0) {
level[e[id].to] = level[u] + 1; q.push(e[id].to); } }
        return level[t] >= 0;
    }
    ll dfs(int u, int t, ll f) {
        if (u == t) return f;
        for (int& i = it[u]; i < (int)g[u].size(); i++) {
            int id = g[u][i]; int v = e[id].to;
            if (e[id].cap > 0 && level[v] == level[u] + 1) {
                ll d = dfs(v, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while (bfs(s, t)) { fill(it.begin(), it.end(), 0); while (ll f = dfs(s, t,
LLONG_MAX)) flow += f; }
        return flow;
    }
};

// Codeforces 1082G "Petya and Graph": pick a vertex set V' maximizing
// (sum of weights of edges with both ends in V') - (sum of a_v over V').
// Project selection: edge = project with profit w, vertex = tool with cost a_v.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m; cin >> n >> m;
    int S = n + m, T = n + m + 1;
    Dinic d(n + m + 2);
    const ll INF = LLONG_MAX / 4;
    for (int v = 0; v < n; v++) { ll a; cin >> a; d.addEdge(v, T, a); }      // pay a_v
if v is taken
    ll total = 0;
    for (int i = 0; i < m; i++) {
        int u, v; ll w; cin >> u >> v >> w; u--; v--;
        total += w;
        d.addEdge(S, n + i, w);          // lose w if edge i is NOT taken
        d.addEdge(n + i, u, INF);        // taking edge i forces taking u and v
        d.addEdge(n + i, v, INF);
    }
    cout << total - d.maxflow(S, T) << "\n";
}
```

7. Common mistakes

- **Negative cycles in MCMF.** SSP assumes none exist initially. Max-profit models with negated costs on cyclic graphs are the usual culprit; check that your network is a DAG or that cycles have non-negative cost.
- **Forgetting the initial potentials** when costs are negative: Dijkstra silently returns wrong paths.
- **Updating potentials of unreachable vertices** with an "infinite" distance — overflow. Only add finite distances.
- **Cost overflow:** total cost = flow × path cost can exceed 32 bits; use `long long`.
- **Hopcroft–Karp without the `dist[u] = -1` dead-end pruning** — correct but can degrade to O(VE).
- **Min-cut with ∞ = `LLONG_MAX`** — overflows when added; use `LLONG_MAX / 4` or the sum of all finite capacities + 1.
- **Wrong cut direction:** the edge u → v is cut when u ∈ S and v ∈ T, not the reverse. Draw the two-vertex case before coding.
- **Lower bounds: checking feasibility against the wrong total** (must equal the sum of positive excesses), or forgetting to delete the t → s edge before the second max-flow (the "extra" flow would cycle through it).
- **Modeling a non-bipartite "different sides" penalty** with a single min cut — it is NP-hard in general (max-cut); look for structure.

8. Practice problems

Review (week 32 level, do quickly)

1. CSES 1694 — Download Speed
2. CSES 1695 — Police Chase (recover the cut)
3. CSES 1711 — Distinct Routes (decompose the flow into paths)

Matching 4. CSES 1696 — School Dance 5. SPOJ MATCHING — Fast Maximum Matching (needs Hopcroft–Karp) 6. Codeforces 1404E — Bricks (König / independent set)

Min-cost flow 7. CSES 2129 — Task Assignment 8. CSES 2130 — Distinct Routes II 9. Kattis — Minimum Cost Maximum Flow (notebook test) 10. Codeforces 863F — Almost Permutation (convex costs) 11. Codeforces 277E — Binary Tree on Plane

Min-cut modeling 12. Codeforces 1082G — Petya and Graph

Lower bounds 13. LibreOJ 115 — feasible circulation with bounds, LibreOJ 116 — max flow with bounds, LibreOJ 117 — min flow with bounds (template checks; statements in Chinese, format is simple) 14. Codeforces 1288F — Red-Blue Graph (min-cost flow with lower bounds)

9. Further reading

- cp-algorithms: Min-cost flow, Flows with demands, Kuhn's algorithm.
- J. E. Hopcroft and R. M. Karp, "An n^{5/2} algorithm for maximum matchings in bipartite graphs", SIAM J. Comput. (1973).
- Ahuja, Magnanti, Orlin, *Network Flows: Theory, Algorithms, and Applications* — chapters on min-cost flow and potentials.
- KACTL: `MinCostMaxFlow`, `hopcroftKarp`, `MinCut` entries.
- Next: Lecture 6 (*Advanced strings*). The World Finals track continues with the Hungarian algorithm, general matching (blossom) and LP duality (worldfinals-03).